

Конспект лекций по курсу: "Алгоритмы и структуры данных"

Титов Василий Андреевич*

Январь, 2026 г.

Содержание

1	Сложность алгоритмов	1
1.1	Проблемы при определении сложности	1
1.2	Упрощения при определении сложности	2
1.3	Оценка сложности с позиции Big O (О-большое)	2
2	Основные структуры данных	2
2.1	Статический массив	2
2.2	Динамический массив	3
2.3	Односвязный список	4
2.4	Двусвязный список	5
2.5	Очередь (queue)	5
2.6	Бинарные деревья (binary tree)	10
2.7	Куча (heap)	14
2.8	Хэш-таблицы	15
3	Алгоритмы сортировки	17
3.1	Медленные сортировки	17
3.1.1	Сортировка пузырьком (bubble sort)	17
3.1.2	Сортировка вставками (insertion sort)	18
3.1.3	Сортировка выбором (selection sort)	20
3.1.4	Сравнение медленных сортировок	20
3.2	Быстрые сортировки	20
3.2.1	Сортировка слиянием (merge sort)	20
3.2.2	Сортировка Хоара	22
4	Дополнительные материалы	22

Аннотация

Собственный конспект лекций по курсу алгоритмы и структуры данных применительно к языку программирования Python.

1 Сложность алгоритмов

1.1 Проблемы при определении сложности

Алгоритм - набор инструкций для решения задачи. При написании программ возникает необходимость оценки качества алгоритма с точки зрения скорости вычислений и объёма потребляемой памяти. Но для одного и того-же кода результат будет зависеть главным образом от следующих параметров:

*Конспект составлен по материалам канала Сергея Балакириева (<https://rutube.ru/channel/43304831/>), конспекта лекций по АиСД 1-го курса ПМИ ФКН ВШЭ Михаила Густокашина (mgustokashin@hse.ru)

- "железа на котором запускается программа
- объёма входных данных

Для исключения влияния первого пункта можно считать количество элементарных операций. При этом вводятся некоторые упрощения. Количество элементарных операций (скорость выполнения программ) или объём потребляемой памяти для n входных данных равно значению функции $T(n)$ (для скорости и памяти могут быть разные функции и значения).

1.2 Упрощения при определении сложности

1.3 Оценка сложности с позиции Big O (О-большое)

$T(n) = O(g(n))$, если, начиная с некоторого N_0 , $\forall n \exists$ некоторая константа c такая, что выполняется равенство $T(n) \leq c \cdot g(n)$. Функция внутри $O(\dots)$ качественно показывает нам, как растёт сложность алгоритма по скорости или памяти при увеличении объёма входных данных.

2 Основные структуры данных

Структура данных - это способ организации хранения и доступа к данным.

2.1 Статический массив

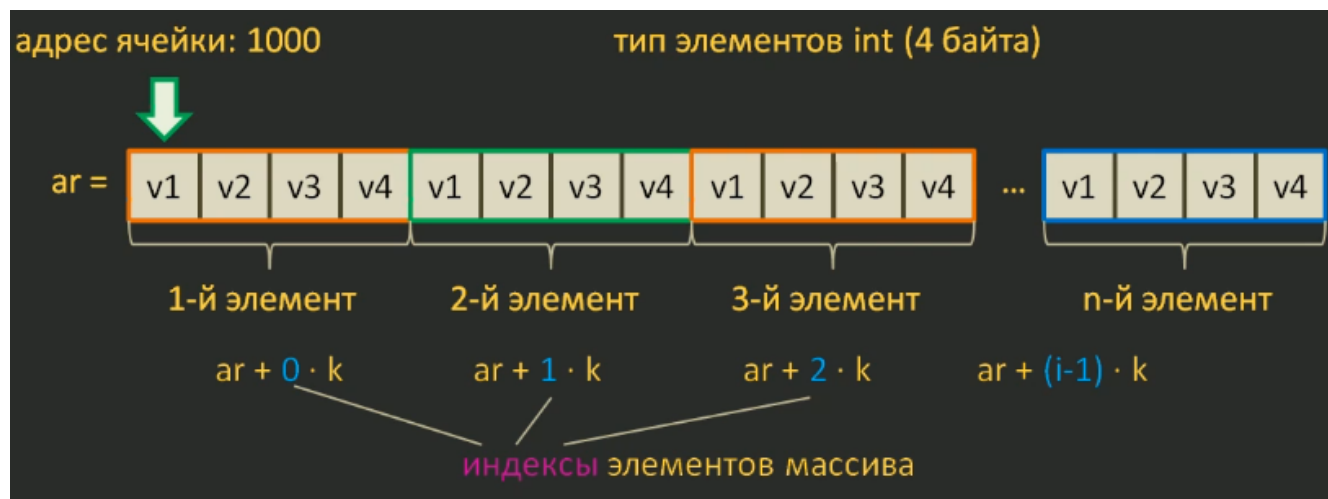


Рис. 1: Структура статического массива

Свойства:

1. Фиксированный размер $\rightarrow$ надо заранее знать n , иначе можем потерять данные.
2. Значения элементов массива можно изменять.
3. Все элементы массива одного типа данных $\rightarrow$ размеры каждого элемента одинаковые.
4. В памяти компьютера каждый элемент находится последовательно друг за другом, начиная с начала массива $\rightarrow$ не всегда эффективно для больших n и при вставке / удалении приходится сдвигать элементы
5. Благодаря свойствам 3 и 4 доступ к i -му элементу статического массива a осуществляется по формуле:

$$a[j] = a + j \cdot k = a[i - 1] = a + (i - 1) \cdot k, \quad (1)$$

где j – индекс i -го элемента; a – адрес 1-го элемента с индексом 0; k – размер одного элемента.

Незаполненные ячейки заполняются "мусором". Счётчик количества ячеек, в которых находится полезная информация, при добавлении или удалении элемента изменяется на +1 или -1 соответственно.

Сложность операций:

1. Чтение и запись: $\Theta(1)$, следует из (1)
2. Вставка и удаление:
 - (a) последнего элемента – $O(1)$
 - (b) i -го элемента – $O(n)$ из-за сдвига промежуточных элементов.

В Python по умолчанию нет такой структуры.

2.2 Динамический массив

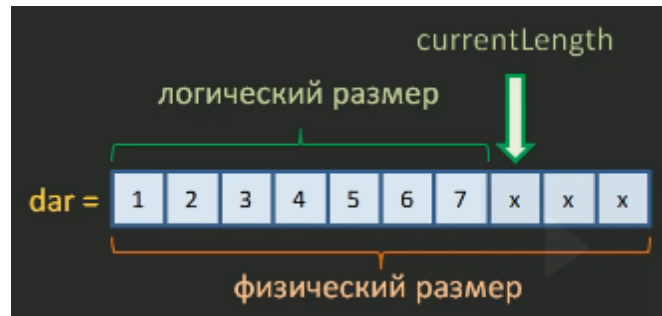


Рис. 2: Структура динамического массива

Свойства:

1. Изменяемый размер $\rightarrow$ если нужен n (логический размер) $>$ capacity (физический размер), то нужно расширить capacity $\rightarrow$ следующая ячейка памяти может быть занята, поэтому перезаписываем данные в другую область памяти. Чтобы уменьшить вероятность повторного перезаписывания обычно в новой области $\text{capacity} = 2 \cdot n$
2. Значения элементов массива можно изменять.
3. Все элементы массива одного типа данных $\rightarrow$ размеры каждого элемента одинаковые. В списках Python для этого все элементы - это ссылки на объекты.
4. В памяти компьютера каждый элемент находится последовательно друг за другом, начиная с начала массива $\rightarrow$ не всегда эффективно для больших n и при вставке / удалении приходится сдвигать элементы.
5. Благодаря свойствам 3 и 4 доступ к i -му элементу динамического массива a осуществляется по формуле:

$$a[j] = a + j \cdot k = a[i - 1] = a + (i - 1) \cdot k, \quad (2)$$

где j – индекс i -го элемента; a – адрес 1-го элемента с индексом 0; k – размер одного элемента.

Незаполненные ячейки заполняются "мусором". Счётчик количества ячеек, в которых находится полезная информация, при добавлении или удалении элемента изменяется на +1 или -1 соответственно.

Сложность операций:

1. Чтение и запись: $\Theta(1)$, следует из (2)
2. Вставка:
 - (a) последнего элемента, если нет переполнения – $O(1)$
 - (b) последнего элемента, если есть переполнение – $O(n)$ из-за перезаписи в другую область памяти.

(с) i -го элемента – $O(n)$ из-за сдвига промежуточных элементов.

3. Удаление:

(а) последнего элемента – $O(1)$

(b) i -го элемента – $O(n)$ из-за сдвига промежуточных элементов.

2.3 Односвязный список

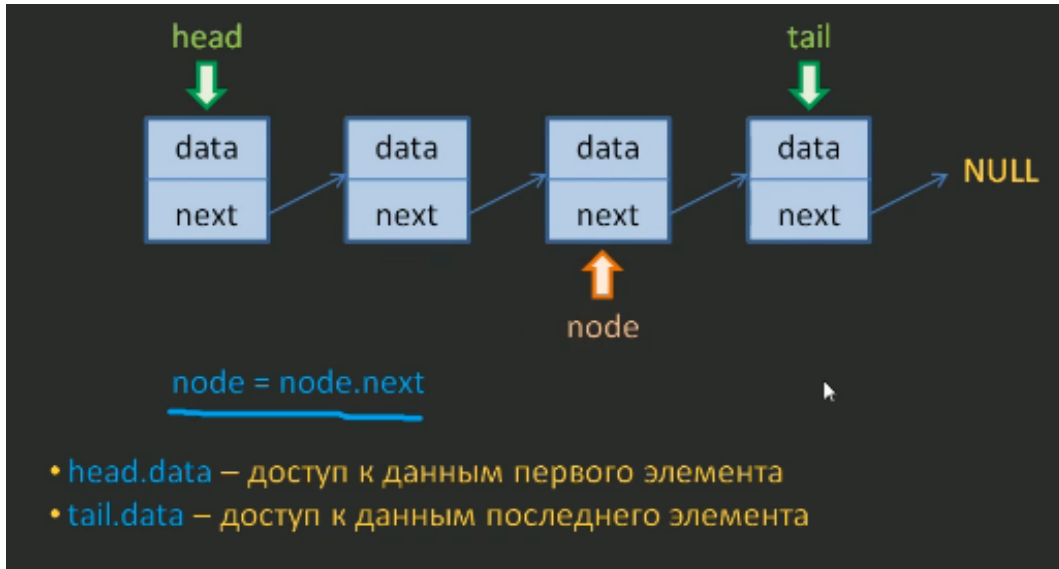


Рис. 3: Структура односвязного списка

Свойства:

1. Изменяемый размер.
2. Значения элементов массива можно изменять.
3. Элементы могут находиться в памяти компьютера в неупорядоченном виде.
4. Каждый элемент содержит ссылку на следующий элемент, а последний на None.
5. Благодаря 3 и 4 можем расширять массив без перезаписи всех элементов при его заполнении.

Храним в памяти адреса начала списка и конца. Если надо проитерироваться, то делаем это от начала.
Сложность операций:

1. Чтение и запись:

(а) последнего и первого элементов – $O(1)$, т.к. храним указатели.

(b) i -го элемента – $O(n)$, т.к. надо последовательно от начала по ссылкам добраться до i -го элемента.

2. Вставка:

(а) последнего и первого элементов – $O(1)$, т.к. храним указатели.

(b) i -го элемента – $O(n)$, т.к. надо последовательно от начала по ссылкам добраться до i -го элемента.

3. Удаление:

(а) первого элемента – $O(1)$

(b) последнего элемента – $O(n)$, т.к. у указателя в конце нет ссылки на предыдущий элемент и надо итерироваться с самого начала.

(с) i -го элемента – $O(n)$ т.к. надо последовательно от начала по ссылкам добраться до i -го элемента.

2.4 Двусвязный список

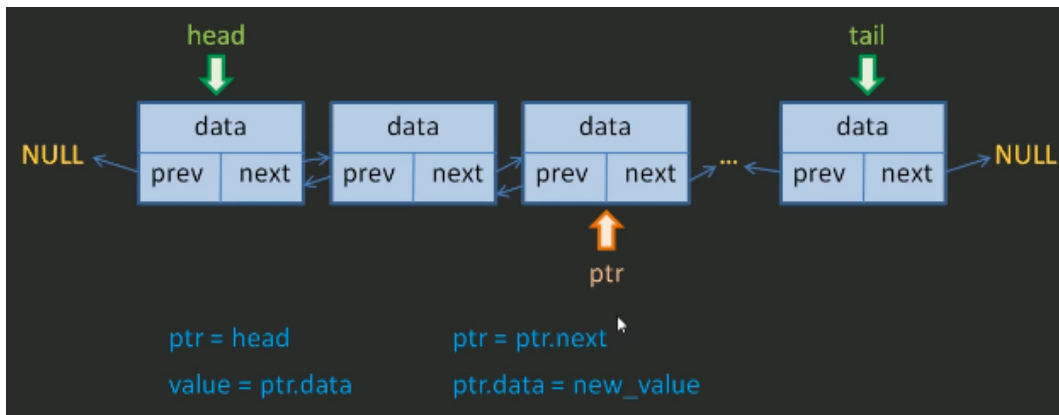


Рис. 4: Структура двусвязного списка

Свойства:

1. Изменяемый размер.
2. Значения элементов массива можно изменять.
3. Элементы могут находиться в памяти компьютера в неупорядоченном виде.
4. Каждый элемент содержит ссылку на следующий и предыдущий элемент, а крайние на None.
5. Благодаря 3 и 4 можем расширять массив без перезаписи всех элементов при его заполнении.
6. Благодаря 4 можем перемещать дополнительный указатель `ptr` в обе стороны → быстро удалять элементы в конце списка.

Храним в памяти адреса начала списка и конца.

Сложность операций:

1. Чтение и запись:
 - (a) последнего и первого элементов – $O(1)$, т.к. храним указатели.
 - (b) i -го элемента – $O(n)$, т.к. надо по ссылкам добраться до i -го элемента.
2. Вставка и удаление:
 - (a) последнего и первого элементов – $O(1)$, т.к. храним указатели и есть ссылки в обе стороны.
 - (b) i -го элемента – $O(n)$, т.к. надо по ссылкам добраться до i -го элемента.

В Python данная структура данных реализуется при помощи библиотеки `collections` и класса `deque`.

2.5 Очередь (queue)

Очередь - это не конкретная структура, как массивы или списки, а абстрактная структура. Суть её заключается в том, что мы работаем в основном с крайними элементами, добавляем или удаляем их. Соответственно, важными оказываются скорость доступа, вставки, удаления именно граничных элементов. Работа с крайними элементами может осуществляться по следующим правилам:

- FIFO (First-In-First-Out) – классическая очередь. Новый элемент добавляется в конец, а извлекается элемент из начала. Используется, например, при хранении заказов от клиентов интернет магазина или буфера данных. Такую очередь можно реализовать на базе двусвязного списка, т.к. мы работаем с обоими концами. Очередь на базе двусвязного списка называют дэк (deque, double ended queue).

- LIFO (Last-In-First-Out) – данный тип очереди ещё часто называют стеком. Хотя понятия стэка и очереди LIFO разные, в стэке мы работаем строго с элементами одного конца без доступа к i -му элементу, а в очереди нет (в основном с двух концов, но можем и обратиться к i -му). Новый элемент добавляется в конец, а извлекается элемент тоже с конца, т.е. работаем с одной стороны. Используется, например, при хранении порядка вызова функций или сохранения предыдущих страничек в браузере, или анализе скобок в выражении. Такую очередь можно реализовать на базе статического/динамического массива (если работаем только с последним элементом), на базе односвязного списка (работаем с первым элементом) или на базе двусвязного списка.

Свойства и сложность операций зависит от того, на базе какой структуры реализована очередь. При этом может использоваться гибридная структура из двусвязного списка и динамического массива. Наличие динамического массива обеспечивает доступ к промежуточному элементу за $O(1)$ внутри элемента двусвязного списка. Но есть и минус: более медленная вставка промежуточных элементов в начало элемента двусвязного списка (т.к. другие промежуточные элементы массива в списке приходится сдвигать) и в конец (т.к. можем уйти в перезапись).

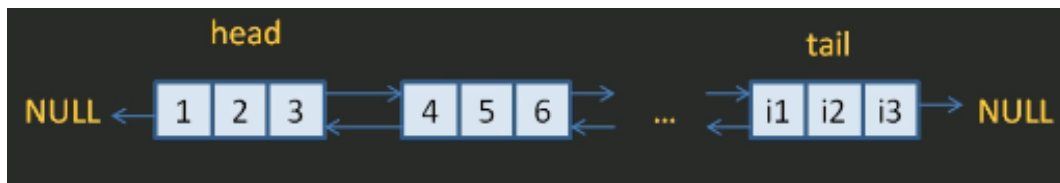


Рис. 5: Структура гибридной очереди

В Python данная структура данных реализуется при помощи библиотеки `collections` и класса `deque`.

Примеры задач Рассмотрим решение нескольких типовых задач на Python.

```
1 from collections import deque
2
3
4 def psp(expression: str) -> bool:
5     '''
6     Функция проверяет, правильно ли расставлены скобки в выражении.
7     Если правильно — возвращает True, если нет — False. Решаем задачу с помощью стэка.
8     '''
9     br_dict = {
10         ')': '(',
11         ']': '[',
12         '}': '{'
13     }
14     stack = deque()
15     for s in expression:
16         if s in br_dict.values():
17             stack.append(s)
18         elif s in br_dict.keys():
19             if not stack:
20                 return f'Есть закрывающая скобка {s}, но нет открывающей {br_dict[s]}'
21             last_br = stack.pop()
22             if last_br != br_dict[s]:
23                 return f'Закрывающая скобка {s} не соответствует открывающей {last_br}'
24     if not stack:
25         return 'OK'
26     else:
27         return f'Незакрытые открывающие скобки: {' '.join(stack)}'
```

```

30 def main():
31     print(psp('3 * (1 + 2)'))
32     print(psp('3 * (1 + 2)'))
33     print(psp('3 * 1 + 2'))
34     print(psp('3 * (1 + 2) / {3 * 2}'))
35     print(psp('3 * (1 + 2) / 3 * 2'))
36     print(psp('3 * (1 + 2) / {3 * 2}'))
37     print(psp('3 * (1 + 2 / 3 * 2)'))
38     print(psp('3 * (1 + 2 / {3 * 2}'))
39     print(psp('3 * (4 * [2 + 10])'))
40     print(psp('3 * (4 * 2 + 10)'))
41     print(psp('3 * (4 * [2 + 10])'))
42
43
44 if __name__ == '__main__':
45     main()

```

```

1 from collections import deque
2
3
4 def oper_push(now, prev, prior_dict) -> bool:
5     '''
6     Функция, которая определяет, надо ли оператор предыдущий оператор выталкивать в
        итоговый стек
7     :param now: текущий оператор
8     :param prev: предыдущий оператор
9     :param prior_dict: словарь с приоритетами
10    '''
11    # операция возведения в степень — правоассоциативная, поэтому её обрабатываем отдельно
12    if now == '**':
13        if prior_dict[prev] > prior_dict[now]:
14            return True
15        return False
16    elif prior_dict[prev] >= prior_dict[now]:
17        return True
18    return False
19
20
21 def shunting_yard(expression: str) -> deque:
22    '''
23    Алгоритм сортировочной станции. Сложность O(N).
24    Функция преобразовывает входное выражение в инфиксной форме в постфиксную.
25    Между операндами должен быть знак пробела.
26    :param expression: выражение
27    '''
28    # словарь с возможными операторами и их приоритетами
29    prior_dict = {
30        '**': 100,
31        '*': 10,
32        '/': 10,
33        '+': 1,
34        '-': 1
35    }
36    # виды скобок
37    br_dict = {
38        ')': '(',
39        ']': '[',

```

```

40         '}' : '{'
41     }
42     # наш итоговый стек
43     stack_OPN = deque()
44     # вспомогательный стек операторов
45     stack_operators = deque()
46     # чтобы было проще парсить элементы заменим скобки на скобки с пробелом.
47     # вдобавок учтём унарный минус добавлением 0, т.е. как бы преобразуем его в бинарный
48     new_expression = ''
49     for s in expression:
50         if s in br_dict.keys() or s in br_dict.values():
51             new_expression += f' {s} '
52         elif s == '-':
53             if new_expression == '':
54                 new_expression += f'0 {s} '
55             elif new_expression[-2] == '(':
56                 new_expression += f'0 {s} '
57             else:
58                 new_expression += s
59         else:
60             new_expression += s
61     # заполняем stack_OPN
62     for el in new_expression.split():
63         if el[0].isdigit():
64             stack_OPN.append(float(el))
65         elif el in prior_dict.keys():
66             # выталкиваем все операторы с большим или равным приоритетом в ответ
67             # останавливаемся, когда дошли до начала stack_operators, до открывающей
68             # скобки или оператора с меньшим приоритетом
69             while stack_operators:
70                 if stack_operators[-1] in br_dict.values():
71                     break
72                 elif oper_push(el, stack_operators[-1], prior_dict):
73                     stack_OPN.append(stack_operators.pop())
74                 else:
75                     break
76             stack_operators.append(el)
77         elif el in br_dict.values():
78             stack_operators.append(el)
79         else:
80             # закрывающая скобка должна выталкивать все элементы в итоговый стек, пока
81             # не встретит открывающую скобку. Поэтому открывающая скобка 100 % должна
82             # быть..
83             while stack_operators[-1] not in br_dict.values():
84                 stack_OPN.append(stack_operators.pop())
85             del stack_operators[-1]
86     # оставшиеся операторы добавляем в итоговый стек
87     while stack_operators:
88         stack_OPN.append(stack_operators.pop())
89     return stack_OPN
90
91 def main():
92     print(shunting_yard('7 - (2 + 3) * 6 + (12 - 3) / 3'))
93     print(shunting_yard('7 - ((2 + 3) * 6 + (12 - 3) / 3)'))
94     print(shunting_yard('6 + 3 * (1 + 4 * 5) * 2'))

```



```

94 print(shunting_yard('1 + 2 ** 3 * 10'))
95 print(shunting_yard('- 2 + 6'))
96
97
98
99 if __name__ == '__main__':
100     main()

```

```

1 from collections import deque
2 from stack_shunting_yard import oper_push, shunting_yard
3
4 def calculate_expression(expression: str) -> float:
5     answer_stack = deque()
6     '''
7     Вычислить значение выражения
8     '''
9     stack_postfix = list(shunting_yard(expression))
10    for el in stack_postfix:
11        if isinstance(el, (int, float)):
12            answer_stack.append(el)
13        else:
14            b = answer_stack.pop()
15            a = answer_stack.pop()
16            answer_stack.append(eval(f'a {el} b'))
17    # return answer_stack[0]
18    return
19
20
21
22 def main():
23     print(calculate_expression('7 - (2 + 3) * 6 + (12 - 3) / 3')) # -20
24     print(calculate_expression('7 - ((2 + 3) * 6 + (12 - 3) / 3)')) # -26
25     print(calculate_expression('6 + 3 * (1 + 4 * 5) * 2')) # 132
26     print(calculate_expression('1 + 2 ** 3 * 10')) # 81
27     print(calculate_expression('- 2 + 6')) # 4
28
29
30
31 if __name__ == '__main__':
32     main()

```

```

1 from collections import deque
2
3
4 def min_in_window(a: list, k: int) -> list:
5     '''
6     Находит минимальный элемент в пределах окна, которое передвигается по массиву
7     :param a: массив
8     :type k: размер окна
9     '''
10    # В дэке хранятся индексы массива a, порядок индексов поддерживается таким,
11    # чтобы значения a[i] в дэке всегда возрастали. Поэтому deq[0] — всегда минимум в окне.
12    deq = deque()
13    for i in range(k):
14        while deq and a[deq[-1]] >= a[i]:
15            deq.pop()
16        deq.append(i)

```

```

17 ans = []
18 ans.append(a[deq[0]])
19 for i in range(k, len(a)):
20     while deq and a[deq[-1]] >= a[i]:
21         deq.pop()
22     while deq and deq[0] <= i - k:
23         deq.popleft()
24     deq.append(i)
25     ans.append(a[deq[0]])
26 return ans
27
28
29 def main():
30     print(min_in_window([1, 3, 2, 4, 5, 3, 1], 3))
31
32
33 if __name__ == '__main__':
34     main()

```

2.6 Бинарные деревья (binary tree)

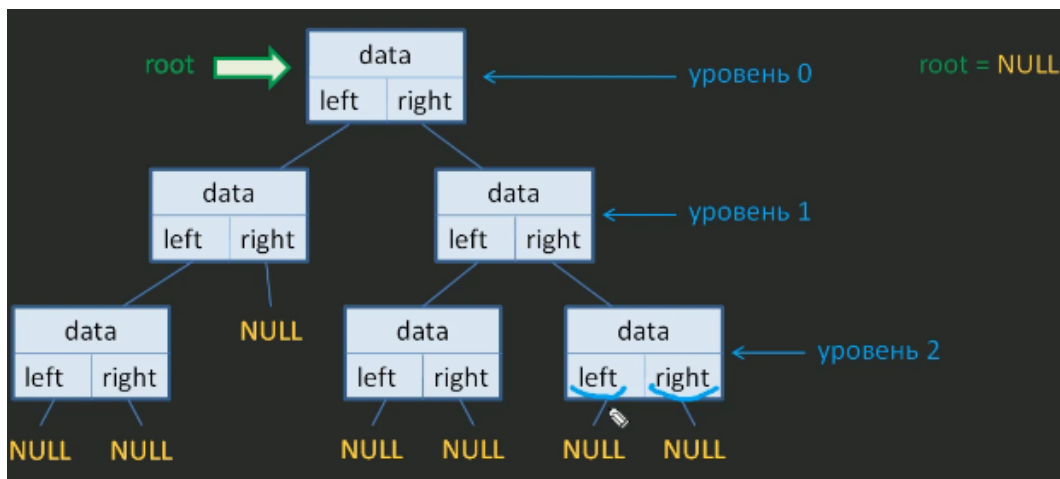


Рис. 6: Структура бинарного дерева

Дерево применяется, когда надо обрабатывать иерархические структуры данных: каталоги или генеологическое дерево.

Бинарное дерево – дерево, у вершин которого не более 2-х потомков

Вершина (node) – узел, у которого есть родитель и потомок.

Корень (root) – узел, у которого отсутствует родитель.

Лист – узел, у которого нет ни одного потомка.

Сбалансированное дерево – дерево, у которого поддеревья от одной вершины отличаются не больше, чем на один уровень.

На вершину дерева ведёт указатель. В каждом узле имеется 3 поля: значение узла (data), ссылки на левого и правого потомков. Иногда хранится ссылка на родителя. На практике стараются свести дерево к бинарному дереву и сбалансировать его.

Для решения определённого класса задач формируют бинарное дерево по следующим правилам:

1. Если добавляемый элемент меньше data в родительском узле, то он уходит в левую ветвь. Если добавляемый элемент больше data в родительском узле, то он уходит в правую ветвь.
2. Если добавляемый элемент равен data в родительском узле, то он не добавляется в дерево.

Способы обхода:

- Обход в ширину (breadth-first). Пробегаясь по каждому узлу на уровне слева направо.
- Обход в глубину. Пробегаясь по каждому узлу сверху вниз. Если сформировали дерево по правилам выше, то при обходе в глубину в порядке LNR всегда будем получать возрастающую последовательность. При проходе в порядке RNL последовательность будет убывающей.

Сложность операций:

1. Поиск:

- (a) В худшем случае, когда n упорядочены: $O(n)$
- (b) На сбалансированном дереве: $O(\log_2 n)$

Методы балансировки двоичных деревьев:

- AVL-дерево;
- красно-чёрное дерево;
- расширяющееся или косое дерево (splay tree)

Удаление вершин:

- Удаляем лист. Просто у родителя убираем ссылку на потомка.
- Удаляем узел a с одним потомком b . Просто у родителя вместо ссылки на удаляемого потомка a вставляем ссылку на b .
- Удаляем узел с двумя потомками. Сам узел не удаляется, а меняется значение. Новое значение – наименьшее значение из правой ветви. В общем случае нужно 3 указателя: на удаляемый узел, на узел с наименьшим значением и на узел-родителя узла с наименьшим значением.

Пример реализации бинарного дерева на Python.

```
1 class Node:
2     '''
3     Класс, моделирующий вершину бинарного дерева. В свойствах self.left и self.right
4     хранятся ссылки на левого и правого потомков.
5     '''
6     def __init__(self, data):
7         self.data = data
8         self.left = self.right = None
9
10
11 class Tree:
12     '''
13     Класс, моделирующий бинарное дерево.
14     '''
15     def __init__(self, value=None):
16         self.root = Node(value)
17
18
19     def __find(self, parent: Node, current: Node, value) -> tuple:
20         '''
21         Метод возвращает узел current со значением value, его родителя и флаг, который
22         говорит, что найдено совпадение value и current.data.
23         Если такого узла нет, то возвращаются два предка parent и current и флаг,
24         который говорит, что не найдено совпадение value и current.data
25         '''
26         if value == current.data:
```

```

25         return parent, current, True
26     elif value < current.data and current.left:
27         return self.__find(current, current.left, value)
28     elif value > current.data and current.right:
29         return self.__find(current, current.right, value)
30     return parent, current, False
31
32
33     def append(self, value):
34         if self.root.data is None:
35             self.root.data = value
36         else:
37             parent, current, flag = self.__find(None, self.root, value)
38             if flag:
39                 print(f'Значение {value} уже есть в дереве')
40                 return
41             elif value < current.data:
42                 current.left = Node(value)
43             else:
44                 current.right = Node(value)
45         print(f'Значение {value} добавлено в дерево')
46
47
48     def __del_list(self, parent: Node, list: Node) -> None:
49         '''
50         Удаление листа
51
52         :param self: Описание
53         :param parent: Описание
54         :param list: Описание
55         '''
56         if parent is None:
57             self.root = None
58         elif parent.left == list:
59             parent.left = None
60         else:
61             parent.right = None
62
63
64     def __find_min(self, parent_min_node: Node, min_node: Node) -> tuple:
65         if min_node.left:
66             return self.__find_min(min_node, min_node.left)
67         else:
68             return parent_min_node, min_node
69
70
71     def __del_node(self, parent: Node, node: Node) -> None:
72         '''
73         Удаление узла с одним потомком
74
75         :param parent: родитель удаляемого узла
76         :param node: удаляемый узел
77         '''
78         if parent is None:
79             if node.left:
80                 self.root = node.left

```

```

81         else:
82             self.root = node.right
83     elif parent.left == node:
84         if node.left:
85             parent.left = node.left
86         else:
87             parent.left = node.right
88     else:
89         if node.left:
90             parent.right = node.left
91         else:
92             parent.right = node.right
93
94
95     def del_val(self, value) -> None:
96         '''
97         Удаление значений из дерева
98
99         :param self: дерево
100        :param value: удаляемое значение
101        '''
102        parent, current, flag = self.__find(None, self.root, value)
103        if not flag:
104            print(f'Значения {value} в дереве нет')
105            return
106        elif current.left == current.right == None:
107            self.__del_list(parent, current)
108        elif current.left and current.right:
109            # ищем минимальное значение в правой ветви
110            parent_min_node, min_node = self.__find_min(current, current.right)
111            # перезаписываем найденный минимум вместо удаляемого значения
112            current.data = min_node.data
113            # удаляем узел с минимумом, это либо лист, либо узел с правым потомком
114            if min_node.right:
115                self.__del_node(parent_min_node, min_node)
116            else:
117                self.__del_list(parent_min_node, min_node)
118        else:
119            self.__del_node(parent, current)
120        print(f'Значение {value} удалено из дерева')
121
122
123     def show_wide(self):
124         '''
125         Показать вершины дерева методом обхода в ширину
126         '''
127         nodes = [self.root]
128         while nodes:
129             if not any(nodes):
130                 break
131             nodes_new_level = []
132             for el in nodes:
133                 if el is None:
134                     print(None, end=' ')
135                 else:
136                     print(el.data, end=' ')

```

```

137         nodes_new_level += [el.left , el.right]
138     nodes = nodes_new_level
139     print()
140
141
142     def show_deep(self , node: Node, reverse: bool=False):
143         '''
144         Показать вершины дерева методом обхода в глубину.
145         При обходе осуществляется сортировка, порядок сортировки определяется параметром
146         reverse.
147         '''
148         if node is None:
149             return None
150         if not reverse:
151             self.show_deep(node.left , reverse)
152             print(node.data)
153             self.show_deep(node.right , reverse)
154         else:
155             self.show_deep(node.right , reverse)
156             print(node.data)
157             self.show_deep(node.left , reverse)
158
159     def main():
160         numbers = [10, 5, 9, 16, 1, 20, 7, 12, 4, 8, 6]
161         t = Tree()
162         for v in numbers:
163             t.append(v)
164         t.show_wide()
165         t.show_deep(t.root)
166         t.show_deep(t.root , reverse=True)
167         t.del_val(100)
168         t.del_val(20)
169         t.show_wide()
170         t.del_val(16)
171         t.show_wide()
172         t.append(6.5)
173         t.show_wide()
174         t.del_val(5)
175         t.show_wide()
176         numbers = [50, 200, 150, 300]
177         t = Tree(100)
178         for v in numbers:
179             t.append(v)
180         t.show_wide()
181         t.del_val(100)
182         t.show_wide()
183
184
185     if __name__ == '__main__':
186         main()

```

2.7 Куча (heap)

Куча представляет собой бинарное дерево. Значение в каждом узле кучи минимумов меньше (для кучи максимумов больше), чем значения каждого потомка. Кучу стараются заполнить плотно.

2.8 Хэш-таблицы

Хэш таблица представляет собой комбинацию динамического массива и специальной хэш-функции $h(k)$. Каждому индексу i массива соответствует пара ключ-значение. Хэш-функция - алгоритм преобразования ключа k в индекс массива i . Порядок заполнения хэш таблицы: ключ $\rightarrow$ некая специальная функция $\rightarrow$ число $k \rightarrow$ хэш-функция $\rightarrow$ индекс массива $\rightarrow$ (ключ-значение). Требования к "хорошей" хэш-функции:

- Детерминированность – одному и тому же ключу k соответствует один и тот же индекс i хэш-таблицы. Разным ключам k соответствуют разные индексы i . Т.к. ключей намного больше, чем m , то для одного и того же ключа могут получаться одинаковые индексы i – коллизия.
- Высокая скорость вычисления i .
- Полученный индекс i должен быть в пределах от 0 до $m - 1$.
- Хэш таблица должна заполняться равномерно, чтобы уменьшить вероятность появления коллизий. В этом случае средняя длина цепочек при коллизии будет равна α .

Есть понятие $\alpha = n/m$ – степень заполненности. n – количество ключей в таблице, m – количество ячеек. $\alpha < 1$ – есть свободные ячейки. $\alpha = 1$ – нет свободных ячеек. $\alpha > 1$ – все ячейки заполнены и есть коллизии.

Методы построения хэш-функции:

- Метод деления: $h(k) = k \% m$. Если m – степень 2, то возникает проблема как на рисунке 7. Чтобы избежать этого эффекта число m нужно выбирать простым и далёким от степени 2 $\rightarrow$ неудобно.

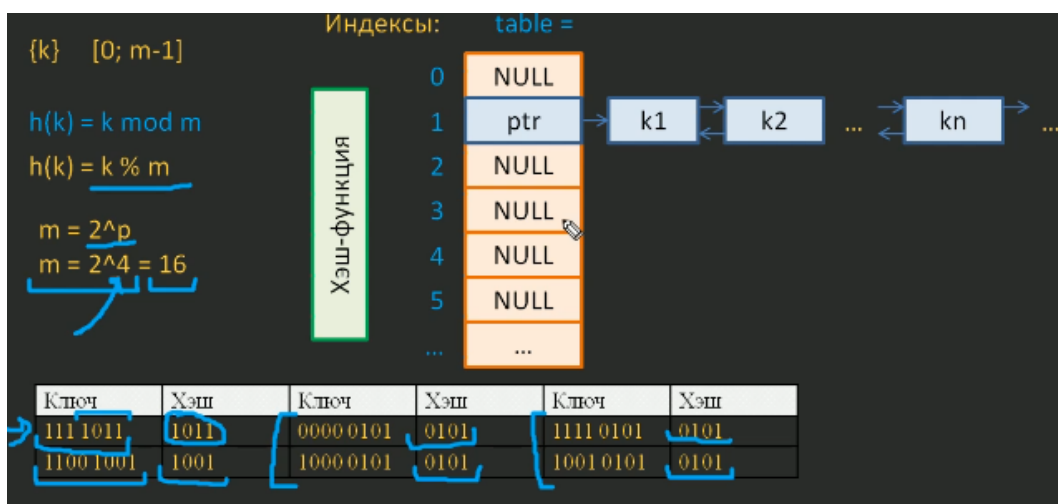


Рис. 7: Иллюстрация недостатка хэш-функции $h(k) = k \% m$

- Метод умножения: $h(k) = \text{floor}(m \cdot [(k \cdot A) \% 1])$, где A – константа от 0 до 1; floor – округление до наименьшего целого. Теперь нет ограничений на m . Но как выбрать A ? Дональд Кнут предложил принять $A = (\sqrt{5} - 1)/2 = 0,61803...$
- SHA – семейство алгоритмов хэширования;
- CRC – циклический избыточный код;
- MD5 – 128-битный алгоритм хэширования.
- Универсальное хэширование

Как бы хорошо не работал алгоритм хэширования для конкретной $h(k)$ можно подобрать такой набор k , что алгоритм хэширования будет давать один и тот же i . Этим могут воспользоваться злоумышленники. Чтобы исключить предсказуемость распределения ключей по индексам таблицы для вновь создаваемой хэш-таблицы используют одну случайную хэш-функцию из набора $\{h_1(k), h_2(k), \dots, h_H(k)\}$. В этом наборе для каждой функции при подаче определённого набора k должны формироваться разные наборы i . Указанный

набор $\{h_1(k), h_2(k), \dots, h_H(k)\}$ называется универсальным. Томас Кормен предложил следующий способ создания универсального набора. Правило, по которому составляется хэш-функция:

$$h_{ab}(k) = ([a \cdot k + b] \% p) \% m, \quad (3)$$

где $p > m$ – простое число, a, b – константы, выбранные случайным образом из наборов $\{1, 2, 3 \dots p - 1\}$, $\{0, 1, 2 \dots p - 1\}$ соответственно.

Хэш функция, построенная по такому правилу, для разных ключей выдаст один и тот же индекс с вероятностью $1/m$. Количество хэш-функций в наборе равно $p(p - 1)$. Значение m выбирается произвольно.

Методы хранения данных в хэштаблицах и способы разрешения коллизий:

- Метод цепочек. Каждый элемент массива содержит ссылку на двусвязный список с парами ключ-значение. Если после работы хэш-функции получился уже занятый индекс, то проваливаемся в двусвязный список и добавляем новую пару ключ-значение. Может получаться длинная цепочка в ячейке. Сложность операций:

1. Доступ:

- (а) Если в ячейке нет коллизии: $O(1)$
- (б) Если в ячейке есть коллизия: $O(1 + \alpha)$.

2. Вставка:

- (а) Коэффициент α меньше порога перезаписи и нет коллизии: $O(1)$;
- (б) Коэффициент α меньше порога перезаписи и есть коллизия: $O(1 + \alpha)$;
- (с) Если уходим в перезапись: $O(n)$. Если при добавлении нового элемента коэффициент α больше порога перезаписи, то:
 - i. Выделяем в памяти в два раза больше ячеек.
 - ii. Перезаписываем уже записанные пары ключ-значение. Это нужно, т.к. размеры таблицы изменились и индексы у существующих данных могут измениться.

3. Удаление:

- (а) Если в ячейке нет коллизии: $O(1)$
- (б) Если в ячейке есть коллизия: $O(1 + \alpha)$

- Метод открытой адресации. В ячейках массива хранятся ссылки на пары ключ-значение. В незаполненных ячейках None. Такой способ применяется в Python.

– Линейное исследование. Формирование хэш-функции:

$$h(k, j) = (h'(k) + j) \% m, \quad (4)$$

где $h'(k)$ – исходная хэш-функция; $h(k)$ – результирующая хэш-функция; j – счётчик, изначально равен 0.

При добавлении нового значения если ячейка занята, то переходим к следующей, увеличивая счётчик j на 1. И так, пока не встретим None. При удалении в ячейки оставляем метку 'deleted'. Это надо, чтобы при поиске ключей за удалённым значением, мы не остановили поиск раньше времени. Могут образовываться длинные цепочки и неравномерность.

– Квадратичное исследование.

$$h(k, j) = (h'(k) + a \cdot j + b \cdot j^2) \% m, \quad (5)$$

где $h'(k)$ – исходная хэш-функция; $h(k, j)$ – результирующая хэш-функция; j – счётчик, изначально равен 0; a, b – константы. Здесь плюс в том, что образуются более короткие последовательности ячеек. Константы a, b, m должны быть подобраны так, чтобы $h(k, j)$ перебирала все ячейки ровно 1 раз.

– Двойное хэширование.

$$h(k, j) = (h'(k) + j \cdot g(k)) \% m, \quad (6)$$

где $h'(k)$ – исходная хэш-функция; $h(k, j)$ – результирующая хэш-функция; j – счётчик, изначально равен 0; $g(k)$ – вспомогательная хэш-функция. Такой способ позволяет хорошо перемешивать ключи. Значения $g(k)$ и m должны быть взаимно простыми, иначе при разрешении коллизии можем

попадать на одни и те же индексы. Томасом Корманом предложена следующая вспомогательная хэш-функция:

$$g(k) = 1 + k \% m', \quad (7)$$

где $m' = m - 1$. Сложность операций при использовании двойного хэширования:

1. Доступ, вставка, удаление в среднем за $O(1)$

В Python по принципу хэш-таблиц работает два типа данных: словарь (dict) и множество (set). Словарь – классическая хэш-таблица. Множество в Python – это та же хэш-таблица, все значения которой равны None. Ключи dict или элементы set должны быть хэшируемыми → неизменяемыми типами данных.

3 Алгоритмы сортировки

3.1 Медленные сортировки

3.1.1 Сортировка пузырьком (bubble sort)

```

1 def sort_bubble(arr: list, reverse: bool=False) -> None:
2     '''
3     Сортирует переданный список arr методом пузырька в порядке возрастания (reverse=False)
4     или в порядке убывания (reverse=True).\n
5     Лучший случай – при сортировке по возрастанию убыванию() массив уже отсортирован по
6     возрастанию убыванию(),
7     количество сравнений = n(n-1)/2, количество обменов = 0. Итоговая сложность: T(n(n-1)
8     /2 + 0) =
9     = T(c1 * n^2 + c2 * n + 0) ~ на| большом количестве n младшие степени отбрасываем| ~
10    T(c1 * n^2) ->
11    -> переходим| к Обольшое| -> O(n^2)\n
12    Худший случай – при сортировке по возрастанию убыванию() массив отсортирован по
13    убыванию возрастанию(),
14    количество сравнений = n(n-1)/2, количество обменов = n(n-1)/2. Итоговая сложность: T(
15    n(n-1)/2 + n(n-1)/2) =
16    = T(n^2 - n) -> O(n^2)\n
17    Сложность по времени: тета(n^2). В лучшем и худшем случаях асимптотическая сложность
18    одинаковая\n
19    Сложность по памяти: O(1). Дополнительная память требуется только для хранения
20    индексов.
21    Сортировка устойчивая.
22    '''
23    n = len(arr)
24    for i in range(n - 1):
25        for j in range(n - 1 - i):
26            if not reverse and arr[j] > arr[j + 1]:
27                arr[j], arr[j + 1] = arr[j + 1], arr[j]
28            elif reverse and arr[j] < arr[j + 1]:
29                arr[j], arr[j + 1] = arr[j + 1], arr[j]
30
31 def sort_bubble_advanced(arr: list, reverse: bool=False) -> None:
32     '''
33     Сортирует переданный список arr методом пузырька в порядке возрастания (reverse=False)
34     или в порядке убывания (reverse=True). Если на итерации счётчика i не было
35     произведено
36     ни одного обмена, то это означает, что массив уже отсортирован и далее выполнять алгоритм
37     не требуется. Благодаря этому, например, для лучшего случая сложность выполнения
38     алгоритма становится O(n).

```

```

31 Но если у нас наихудший случай, то на каждой итерации мы будем дополнительно
32 проверять наш флаг, это
33 дополнительно  $n - 1$  операция. Константа увеличится, но сложность также останется  $O(n^2)$ 
34 . Поэтому
35 алгоритм выгоден, когда наш массив полностью или частично отсортирован.
36 '''
37 n = len(arr)
38 for i in range(n - 1):
39     swapped = False
40     for j in range(n - 1 - i):
41         if not reverse and arr[j] > arr[j + 1]:
42             arr[j], arr[j + 1] = arr[j + 1], arr[j]
43             swapped = True
44         elif reverse and arr[j] < arr[j + 1]:
45             arr[j], arr[j + 1] = arr[j + 1], arr[j]
46             swapped = True
47     if not swapped:
48         break
49 def main():
50     unsorted_list = [8, 5, -10, 11, 0]
51     sort_bubble(unsorted_list)
52     print(unsorted_list)
53
54     unsorted_list = [8, 5, -10, 11, 0]
55     sort_bubble(unsorted_list, True)
56     print(unsorted_list)
57
58
59 if __name__ == '__main__':
60     main()

```

3.1.2 Сортировка вставками (insertion sort)

```

1 def sort_insertion(arr: list, reverse: bool=False) -> None:
2     '''
3     Сортирует переданный список arr методом вставки в порядке возрастания (reverse=False)
4     или в порядке убывания (reverse=True).\n
5     Лучший случай — при сортировке по возрастанию убыванию() массив уже отсортирован по
6     возрастанию убыванию(),
7     количество сравнений =  $n - 1$ , количество обменов = 0. Итоговая сложность:  $T(n - 1 + 0)$ 
8     -> переходим к Обольшое-|
9     ->  $O(n) \cdot n$ 
10    Худший случай — при сортировке по возрастанию убыванию() массив отсортирован по
11    убыванию возрастанию(),
12    количество сравнений =  $n(n - 1) / 2$ , количество обменов =  $n(n - 1) / 2$ . Итоговая
13    сложность:
14     $T(n(n-1)/2 + n(n-1)/2) = T(n^2 - n) \sim$  на| большом количестве  $n$  младшие степени
15    отбрасываем|  $\sim T(n^2) \rightarrow$ 
16    -> переходим к Обольшое-| ->  $O(n^2) \cdot n$ 
17    Сложность по времени:  $O(n^2)$ . В лучшем и худшем случаях асимптотическая сложность
18    разная\n
19    Сложность по памяти:  $O(1)$ . Дополнительная память требуется только для хранения
20    индексов.
21    Алгоритм хорошо себя показывает в полностью или частично отсортированном массиве, а
22    также когда надо

```

```

15 доотсортировать вновь добавленную часть массива.
16 '''
17 n = len(arr)
18 for i in range(1, n):
19     for j in range(i, 0, -1):
20         if not reverse and arr[j] < arr[j - 1]:
21             arr[j], arr[j - 1] = arr[j - 1], arr[j]
22         elif reverse and arr[j] > arr[j - 1]:
23             arr[j], arr[j - 1] = arr[j - 1], arr[j]
24         else:
25             break
26
27
28 def sort_insertion_advanced(arr: list, reverse: bool=False) -> None:
29     '''
30     Сортирует переданный список arr методом вставки в порядке возрастания (reverse=False)
31     или в порядке убывания (reverse=True). Преимущество перед обычной версией и
32     sort_bubble в том, что у нас операция обмена заменяется на операцию присваивания обмен( — это пара
33     присваиваний). Текущий элемент now вставляется в нужное место только один раз, после go2- цикла.
34     Благодаря этому сокращается число операций присваивания, которая в современном компьютере достаточно
35     медленная по сравнению с чтением данных.
36     Сортировка устойчивая.
37     '''
38     n = len(arr)
39     for i in range(1, n):
40         now = arr[i]
41         new_place = 0
42         for j in range(i, 0, -1):
43             if not reverse and now < arr[j - 1]:
44                 arr[j] = arr[j - 1]
45             elif reverse and now > arr[j - 1]:
46                 arr[j] = arr[j - 1]
47             else:
48                 new_place = j
49                 break
50         arr[new_place] = now
51
52
53 def main():
54     unsorted_list = [8, 5, -10, 11, 0]
55     sort_insertion(unsorted_list)
56     print(unsorted_list)
57
58     unsorted_list = [8, 5, -10, 11, 0]
59     sort_insertion(unsorted_list, True)
60     print(unsorted_list)
61
62     unsorted_list = [8, 5, -10, 11, 0]
63     sort_insertion_advanced(unsorted_list)
64     print(unsorted_list)
65
66     unsorted_list = [8, 5, -10, 11, 0]

```

```

67 sort_insertion_advanced(unsorted_list, True)
68 print(unsorted_list)
69
70
71
72
73 if __name__ == '__main__':
74     main()

```

3.1.3 Сортировка выбором (selection sort)

```

1 def sort_selection(arr: list, reverse: bool=False) -> None:
2     '''
3     Сортировка выбором.
4     Сложность по времени: O(N^2)
5     Сложность по памяти: O(1)
6     Не устойчивая.
7     '''
8     n = len(arr)
9     for i in range(n - 1):
10        indx_min = i
11        for j in range(i + 1, n):
12            if not reverse and arr[j] < arr[indx_min]:
13                indx_min = j
14            elif reverse and arr[j] > arr[indx_min]:
15                indx_min = j
16        arr[i], arr[indx_min] = arr[indx_min], arr[i]
17
18
19 def main():
20     unsorted_list = [8, 5, -10, 11, 0]
21     sort_selection(unsorted_list)
22     print(unsorted_list)
23
24     unsorted_list = [8, 5, -10, 11, 0]
25     sort_selection(unsorted_list, True)
26     print(unsorted_list)
27
28
29 if __name__ == '__main__':
30     main()

```

3.1.4 Сравнение медленных сортировок

Сортировка называется устойчивой, если она сохраняет взаимный порядок одинаковых элементов.

3.2 Быстрые сортировки

3.2.1 Сортировка слиянием (merge sort)

```

1
2 def sort_merge(lst: list | tuple, reverse: bool=False) -> list:
3     '''
4     Возвращает список, отсортированный методом слияния.
5     Сложность по времени: O(n*log(2)n)
6     Сложность по памяти: O(n)
7     '''
8     if (length := len(lst)) < 2:

```

Свойство	Пузырьком	Вставками	Выбором
Худший случай	$O(N^2)$	$O(N^2)$	$O(N^2)$
Лучший случай	$O(N^2)$	$O(N)$	$O(N^2)$
Кол-во присваиваний	$O(N^2)$	$O(N^2)$	$O(N)$
Устойчивая	Да	Да	Нет

При этом сортировка пузырьком обладает важным свойством: обмены происходят только между соседними элементами, что может быть полезно при некоторых дополнительных ограничениях.

В случае, если нам нужно минимизировать количество присваиваний и нам не важна устойчивость, можно использовать сортировку выбором.

В реальности же из квадратичных сортировок используется сортировка вставками, т.к. он отлично работает на почти отсортированных данных, которые достаточно часто возникают при решении реальных задач.

Рис. 8: Сравнение квадратичных сортировок

```

9         return lst
10    else:
11        mid = length // 2
12        a, b = sort_merge(lst[mid:], reverse), sort_merge(lst[:mid], reverse)
13    return merge(a, b, reverse)
14
15
16    def merge(a: list | tuple, b: list | tuple, reverse: bool=False) -> list:
17        '''
18        Возвращает отсортированный список, полученный слиянием отсортированных массивов a и b.
19        '''
20        merged_list = []
21        i, j = 0, 0
22        length_a, length_b = len(a), len(b)
23        while i < length_a and j < length_b:
24            if not reverse:
25                if a[i] < b[j]:
26                    merged_list.append(a[i])
27                    i += 1
28            else:
29                merged_list.append(b[j])
30                j += 1
31        else:
32            if a[i] > b[j]:
33                merged_list.append(a[i])
34                i += 1
35            else:
36                merged_list.append(b[j])
37                j += 1
38        merged_list += a[i:] + b[j:]
39    return merged_list
40
41
42    def main():
43        sorted_list_1 = [5, 6, 7, 8, 20]
44        sorted_list_2 = [4, 5, 6, 10]
45        sorted_list_3 = [20, 8, 7, 6, 5]
46        sorted_list_4 = [10, 6, 5, 4]
47        print(merge(sorted_list_1, sorted_list_2))

```

```
48 print(merge(sorted_list_3, sorted_list_4, True))
49
50 unsorted_list = [8, 5, -10, 11, 0]
51 print(sort_merge(unsorted_list))
52 print(sort_merge(unsorted_list, True))
53
54
55 if __name__ == '__main__':
56     main()
```

3.2.2 Сортировка Хоара

4 Дополнительные материалы

ВШЭ, лекции Густокашина ФПМИ МФТИ Яндекс хендбук, Яндекс тренировки Искусство программировать, Дональд Кнут Алгоритмы: построение и анализ, Томас Кормен